

XIAMEN
UNIVERSITY

COM102 – Programming Language C

Chapter 5 – Pointers and Arrays (ii) – Lab

Dr. Najla'a A. M. Draib

5.3 Accessing Array Elements

- E.g.
  - Specifying an Array's Size with a Symbolic Constant and Initializing Array Elements with Calculations

```
1  // Fig. 6.5: fig06_05.c
2  // Initializing the elements of array s to the even integers from 2 to 10.
3  #include <stdio.h>
4  #define SIZE 5 // maximum size of array
5
6  // function main begins program execution
7  int main(void)
8  {
9     // symbolic constant SIZE can be used to specify array size
10    int s[SIZE]; // array s has SIZE elements
11
12    for (size_t j = 0; j < SIZE; ++j) { // set the values
13       s[j] = 2 + 2 * j;
14    }
15
16    printf("%s%13s\n", "Element", "Value");
17
18    // output contents of array s in tabular format
19    for (size_t j = 0; j < SIZE; ++j) {
20       printf("%7u%13d\n", j, s[j]);
21    }
22 }
```

```
Element         Value
      0             2
      1             4
      2             6
      3             8
      4            10
```

Solution

- **EX**
  - Summing the Elements of an Array

```c
// Fig. 6.6: fig06_06.c
// Computing the sum of the elements of an array.
#include <stdio.h>
#define SIZE 12

// function main begins program execution
int main(void)
{
   // use an initializer list to initialize the array
   int a[SIZE] = {1, 3, 5, 4, 7, 2, 99, 16, 45, 67, 89, 45};
   int total = 0; // sum of array

   // sum contents of array a
   for (size_t i = 0; i < SIZE; ++i) {
      total += a[i];
   }

   printf("Total of array element values is %d\n", total);
}
```

```
Total of array element values is 383
```

Examples

Find the Largest Element in an Array

- Solution

```c
#include <stdio.h>

int main() {
    int numbers[5] = {4, 2, 9, 1, 5};
    int largest = numbers[0];

    // Find the largest element
    for (int i = 1; i < 5; i++) {
        if (numbers[i] > largest) {
            largest = numbers[i];
        }
    }

    printf("Largest element: %d\n", largest);  // Output: 9

    return 0;
}
```

5.4 Character Arrays & Strings in C

Character arrays

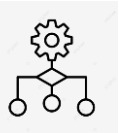
- A C “string” is a **character array** ending with **'\0’**

```
char name[20];                  // declaration
char name[] = "Hello";          // initialization
```

```
char string1[] = "first";
```

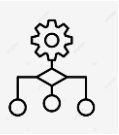
```
char string1[] = {'f', 'i', 'r', 's', 't', '\0'};
```

5.4 Character Arrays & Strings in C



- **Key properties**
  - Stores sequences of characters: names, messages, etc.
  - Must reserve enough space for all characters **plus** the '\0' terminator
- **Basic string.h functions**
  - strlen(s) – length of string s
  - strcpy(dest, src) – copy src into dest
  - strcmp(s1, s2) – compare two strings (0 if equal)

5.4. Character Arrays



1. strlen() – String Length

- **Format:** size_t strlen(const char *str);
- **Description:** Returns the number of characters **before** '\0' (does **not** count the null terminator).
- **Example:**

```
char name[] = "Najla";
int len = strlen(name);    // len = 5
```

2. strcpy() – Copy String

- **Format:** char *strcpy(char *dest, const char *src);
- **Description:** Copies src (including '\0') into dest. dest must have **enough space**.
- **Example:**

```
char src[]  = "Hello";
char dest[10];

strcpy(dest, src);          // dest now contains "Hello"
```

5.4. Character Arrays

strcmp() – Compare Strings

- **Format:** int strcmp(const char *s1, const char *s2);
- **Description:** Compares s1 and s2 in dictionary (ASCII) order.
  - Returns **0** → strings are equal
  - Returns **< 0** → s1 comes **before** s2
  - Returns **> 0** → s1 comes **after** s2
- **Example:**

```
int r = strcmp("apple", "banana");   // r < 0
```

5.4 Initializing a Character Array

- E.g.
  - Demonstrating Character Arrays

```c
#include <stdio.h>
#include <string.h>

#define MAX_LEN 20

int main(void)
{
    char word1[MAX_LEN];
    char word2[MAX_LEN];
    char copy[MAX_LEN];
    char label[] = "Comparison result";
    int cmp;

    // Input
    printf("Enter first word (max %d characters): ", MAX_LEN - 1);
    scanf("%19s", word1);

    printf("Enter second word (max %d characters): ", MAX_LEN - 1);
    scanf("%19s", word2);

    // Compare strings
    cmp = strcmp(word1, word2);

    printf("\n%s: ", label);
    if (cmp == 0) {
        printf("the words are equal.\n");
    } else if (cmp < 0) {
        printf("\"%s\" comes before \"%s\" in dictionary order.\n", word1, word2);
    } else {
        printf("\"%s\" comes after \"%s\" in dictionary order.\n", word1, word2);
    }

    // Copy first word into copy
    strcpy(copy, word1);
    printf("Copy of first word: %s\n", copy);

    // Print original words
    printf("You entered: word1 = %s, word2 = %s\n", word1, word2);

    return 0;
}
```

5.4. Character Arrays

- Demonstrating Character Arrays
  - E.g.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char source[] = "Hello";
    char destination[6];

    strcpy(destination, source);

    printf("Copied string: %s\n", destination);  // Output: Hello

    return 0;
}
```

Exercise: Simple String Analyzer

Task:
Write a C program that reads one word from the user and prints some information about it.

Requirements:
1. Declare:

```c
#define SIZE 30

char text[SIZE];           // to store the user input
char label[] = "Result";   // initialized using a string literal
```

2. **Ask** the user to enter **one word** (no spaces) with at most 29 characters, and read it using:

```c
scanf("%29s", text);
```

3. Print the original word, for example:

```
Result: Programming
```

Exercise: Simple String Analyzer

4. Using **one for loop** that stops when it reaches **'\0':**

- **Print** each character with its index, e.g.
  index 0: h
  index 1: e …

```
index 0: P
index 1: r
index 2: o
index 3: g
index 4: r
index 5: a
index 6: m
index 7: m
index 8: i
index 9: n
index 10: g
```

You can complete this code:

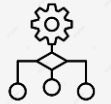
```
for (i = 0; text[i] != '\0'; i++) {
    printf(......)
    length++;

    if (.....) {
        countA++;
    }
}
```

5. At the same time, **count**:
   - the total **length** of the word
   - how many times the letter **'a'** or **'A'** appears
- After the loop, **print** a summary like:

```
Length = 11, number of 'a' or 'A' = 1
```

Solution



```c
#define SIZE 30   // maximum characters including '\0'

int main(void)
{
    char text[SIZE];           // to store the user input
    char label[] = "Result";   // initialized using a string literal
    int i;
    int length = 0;
    int countA = 0;

    // 1. Read one word
    printf("Enter one word (max %d characters): ", SIZE - 1);
    scanf("%29s", text);       // read at most 29 characters + '\0'

    // 2. Print the original word
    printf("\n%s: %s\n\n", label, text);

    // 3. One loop: print each character and count length and 'a'/'A'
    for (i = 0; text[i] != '\0'; i++) {
        printf("index %d: %c\n", i, text[i]);
        length++;

        if (text[i] == 'a' || text[i] == 'A') {
            countA++;
        }
    }
    printf("\nLength = %d, number of 'a' or 'A' = %d\n", length, countA);
    return 0;
}
```



Thank You

Q & A